

Note dal Codice C#

Informazioni Generali

- **Versioni .NET:** Versione pari = 3 anni supporto | Versione dispari = 18 mesi supporto
- **File essenziali:** `.cs` (codice sorgente) e `.csproj` (configurazione progetto)
- **Comando terminale:** `dotnet` per creare, compilare ed eseguire progetti

Configurazione Progetto

Nel file `.csproj` rimuovere per semplicità:

- `<ImplicitUsings>enable</ImplicitUsings>` ' Se non vuoi che il compilatore aggiunga automaticamente `using System`
- `<Nullable>enable</Nullable>` ' Se vuoi gestire manualmente i null pointer

Problemi Comuni

NAME COLLISIONS

- **Problema:** Due classi con lo stesso nome in namespace diversi
- **Soluzione:** Usare i namespace

FENCE POST ERROR (Off by One Error)

- **Descrizione:** Errore nei loop causato da condizioni scorrette
- **Esempio:** `for (int i = 0; i <= 10; i++)` VS `for (int i = 0; i < 10; i++)`
- **Attenzione:** Verificare sempre le condizioni di uscita!

Namespace Essenziali

Namespace	Uso
<code>using System;</code>	Classi fondamentali (Console, etc.)
<code>using System.Linq;</code>	Operatori LINQ per manipolare sequenze

Stili di Codice

1TBS Style (1 True Brace Style)

```
for (int i = 1; i <= 10; i++) {  
    Console.Write(i);  
}
```

- Parentesi graffe sulla stessa riga

Loop: For vs While

Tipo	Uso Ideale
For	Quando sai il numero di iterazioni
While	Quando il numero di iterazioni è variabile

```
// For: controllo preciso  
for (int i = 1; i <= 10; i++) { ... }  
  
// While: condizione variabile  
while (condizione) { ... }
```

Tipi di Funzioni

Predicate (Funzione 'Bool)

Restituisce true/false per filtrare dati

```
bool IsEven(int a) => a % 2 == 0;
```

Transformer-Function (Funzione 'Output Diverso)

Trasforma un input in un output di tipo diverso

```
int Square(int a) => a * a;
```

Sintassi Modern C# (Arrow Syntax)

Ligature `=>`: Sostituisce `return`

```
// VECCHIO stile
bool IsEven(int a) {
    if (a % 2 == 0) return true;
    else return false;
}

// MODERNO stile (Arrow)
bool IsEven(int a) => a % 2 == 0;
```

LINQ - Language Integrated Query

Concetto: Operatori LINQ accettano funzioni come parametri

Operatori Principali

Operatore	Funzione
<code>Range()</code>	Genera sequenza di interi
<code>Where()</code>	Filtra dati (Predicate)
<code>Select()</code>	Trasforma ogni elemento (Transformer)
<code>ToList()</code>	Converte in lista (materializza i dati)
<code>ForEach()</code>	Itera su ogni elemento

Esempio Compositional Style

```
Enumerable.Range(1, 10)           // Genera: 1, 2, 3, ..., 10
.Where(a => a % 2 == 0)           // Filtra: 2, 4, 6, 8, 10
.Select(a => a * a)               // Trasforma: 4, 16, 36, 64, 100
.ToList()                        // Salva in memoria
.ForEach(Console.WriteLine);     // Stampa ogni elemento
```

Lazy Evaluation vs Eager Evaluation

- **Lazy Evaluation** (Enumerable): Dati generati uno alla volta (generatore)
- **Eager Evaluation** (ToList()): Tutti i dati generati in memoria subito

```
// LAZY: Enumerable genera uno alla volta
Enumerable.Range(1, 10).Where(a => a % 2 == 0)

// EAGER: ToList() materializza tutto
Enumerable.Range(1, 10).Where(a => a % 2 == 0).ToList()
```

LAMBDA FUNCTION

Definizione: Funzione anonima (senza nome) definita inline nel punto di utilizzo

```
// Lambda semplice
a => a % 2 == 0

// Lambda con blocco
a => {
    int result = a * a;
    return result;
}
```

Uso: Passata come parametro agli operatori LINQ

Compositional Style

Concetto: Composizione di operazioni sequenziali su dati

- Ogni operatore riceve il risultato del precedente
- I dati "fluiscono" attraverso le operazioni

```
Input ' Range ' Where ' Select ' ToList ' ForEach ' Output
```

Classe Enumerable

- **Definizione:** Classe che contiene metodi per creare e manipolare sequenze
- **Caratteristica:** Genera dati uno alla volta (lazy evaluation)
- **Vantaggio:** Efficiente in memoria per sequenze grandi

Buone Pratiche

1. Usare **Arrow Syntax** per funzioni semplici
2. Preferire **For-loop** quando conosci il numero di iterazioni

3. Preferire **LINQ** per trasformazioni di sequenze
4. Usare **namespace** per evitare collisioni di nomi
5. Usare **predicate** per filtering
6. Usare **transformer** per mapping

Esempio Completo: Procedural vs Modern

Procedural Style

```
for (int i = 1; i <= 10; i++) {  
    if (IsEven(i)) {  
        Console.WriteLine(Square(i));  
    }  
}  
  
bool IsEven(int a) => a % 2 == 0;  
int Square(int a) => a * a;
```

Modern Style (Compositional)

```
Enumerable.Range(1, 10)  
    .Where(a => a % 2 == 0)  
    .Select(a => a * a)  
    .ForEach(Console.WriteLine);
```

Risultato identico, codice più elegante e funzionale!